

claude-windows / c6028e72-ebfd-4218-89a8-c4d7dd23be96

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c6028e72-ebfd-4218-89a8-c4d7dd23be96\subagents\agent-a9e6d21081ff4b03b.jsonl`
- SHA-256: `7b4abc62407dafcc442357e8f3ca8a88bbdfacbbf41c167ffb31f363636e06`
- Source modified: `2026-06-08T10:25:11+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `subagents`
- session_id: `c6028e72-ebfd-4218-89a8-c4d7dd23be96`

Transcript

1. user (2026-06-08T10:22:54.195Z)

Adversarially review a small PR in the sports-data-collector repo. Your job is to find REAL bugs and FACTUAL inaccuracies only ? be skeptical, but do not invent issues. Return confirmed findings with severity.

Files to review (read them in full):

1. C:/Users/avidu/Projects/sports-data-collector/src/cli/normalize_annotations.py (the new normalizer)
2. C:/Users/avidu/Projects/sports-data-collector/tests/test_normalize_annotations.py
3. C:/Users/avidu/Projects/sports-data-collector/docs/MANUAL_INGESTION_POSITIONING.md

Context: the normalizer is a standalone pre-import step that cleans a VLC rally-annotator CSV so `import-local` (C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py::_parse_local_annotations) ingests it safely. Two landmines it must defend against: (a) blank or end<=start end_time makes safe_float_required raise and abort the whole import (src/core/parsing.py); (b) fractional/duplicate rally_number truncates via int(float()) (src/core/parsing.py::safe_int) and collides on the (video_id, rally_number) PK, where db.insert_badminton_rallies does DELETE-then-INSERT (src/core/db.py).

Do these checks:

- A. CORRECTNESS of normalize_annotations.py: trace the logic for edge cases ? empty CSV, header-only, a `rally`-only (rich) file with no rally_number, negative start_time, rows out of start order, duplicate (start,end), a value like "abc" in a time field, the renumbered/fractional_ids/duplicate_ids report flags. Does it ever (i) let a fractional/duplicate id reach the output rally_number, (ii) keep a blank/reversed-end row, (iii) crash on malformed input? Confirm by reading the code, not guessing.
- B. INTEGRATION: does the output CSV's columns/values actually feed _parse_local_annotations safely (read curate.py:232-298 and src/core/parsing.py)? Will extra columns (original_rally_number, notes, shots_count) break anything? Will the renumbered integer ids avoid the PK collision in db.py?
- C. DOC ACCURACY: spot-check every code reference in MANUAL_INGESTION_POSITIONING.md against the actual code (function names, the claim that load_golden_gts reads only start_time/end_time, that the export is start,end intervals-only, that CurationStatus is STAGING/CURATED/REJECTED, that import-local hardcodes CURATED). Flag any claim not supported by the code. Also check the indexer file C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/eval_partial_labels.py for the restrict_to_window / max(end_time) claims.

Return a concise list: each finding = {severity: blocker|major|minor|nit, file, what, why it's wrong, suggested fix}. If something is correct, do NOT list it. End with a one-line overall verdict (ship / fix-first).

2. assistant (2026-06-08T10:22:56.925Z)

I'll review this PR adversarially. Let me start by reading all the files in full.

3. user (2026-06-08T10:22:57.882Z)

```

1      """Normalize a VLC rally-annotator CSV into a clean lean CSV that ``import-local``
2      ingests safely.
3
4      Two verified landmines in PARTIAL human annotation data (a video tagged only up to
5      some point, with mid-session corrections):
6
7          1. A blank or ``end_time <= start_time`` row makes ``safe_float_required`` RAISE,
8             which aborts the WHOLE import (one bad row kills the entire video, curate.py
9             _parse_local_annotations). -> we DROP such rows, mirroring the downstream
10             indexer loader which already skips ``not end > start``.
11          2. A fractional or duplicate ``rally_number`` (e.g. ``7.5`` inserted for a rally
12             the annotator missed) truncates via ``int(float())`` and COLLIDES on the
13             ``(video_id, rally_number)`` primary key; the DELETE-then-INSERT upsert then
14             silently destroys one rally. -> we RENUMBER 1..N by start_time, recording the
15             original id for traceability.
16
17      It also accepts mm:ss / HH:MM:SS times (converted to decimal seconds) and folds a
18      pipe-delimited ``ending_reason`` ('forced_error|other') to its primary token, so the
19      output is always the canonical lean schema::
20
21          rally_number,start_time,end_time,ending_reason,sport[,shots_count]
22
23      plus traceability columns (``original_rally_number``, ``notes``) that import-local
24      ignores. Standalone on purpose: it does NOT touch the DB schema or the
25      rally_number-ordered export. Run it, then feed the output to ``curate import-local``.
26
27      python -m src.cli.normalize_annotations --in "Match.rallies.csv" --out clean.csv
28      """
29      from __future__ import annotations
30
31      import argparse
32      import csv
33      import os
34      from dataclasses import dataclass, field
35      from typing import Any, Dict, List, Optional, Tuple
36
37      # Canonical lean output columns + traceability (import-local reads the first four/
38      # five; original_rally_number + notes are ignored by the importer but kept for humans).
39      OUT_FIELDS = ["rally_number", "start_time", "end_time", "ending_reason", "sport",
40                   "shots_count", "original_rally_number", "notes"]
41
42
43      @dataclass
44      class NormalizeReport:
45          rows_in: int = 0
46          rows_out: int = 0
47          dropped: List[Dict[str, Any]] = field(default_factory=list) # {orig_id, reason,
start, end}
48          renumbered: bool = False
49          fractional_ids: List[str] = field(default_factory=list)
50          duplicate_ids: List[str] = field(default_factory=list)
51          mmss_converted: int = 0
52
53          def summary(self) -> str:
54              lines = [f"rows in: {self.rows_in} -> rows out: {self.rows_out}"]
55              if self.dropped:
56                  lines.append(f"dropped {len(self.dropped)} bad row(s):")
57                  for d in self.dropped:
58                      lines.append(f"  - orig id {d['orig_id']!r}: {d['reason']} "
59                                  f"(start={d['start']!r}, end={d['end']!r})")
60              if self.renumbered:
61                  extra = []
62                  if self.fractional_ids:
63                      extra.append(f"fractional ids {self.fractional_ids}")
64                  if self.duplicate_ids:

```

```

65         extra.append(f"duplicate ids {self.duplicate_ids}")
66         lines.append("renumbered rallies 1..N by start_time"
67                     + (f" ({'; '.join(extra)})" if extra else ""))
68         if self.mmss_converted:
69             lines.append(f"converted {self.mmss_converted} mm:ss timestamp(s) to
seconds")
70         if not self.dropped and not self.renumbered and not self.mmss_converted:
71             lines.append("clean input ? no changes beyond column normalization")
72         return "\n".join(lines)
73
74
75     def _to_seconds(raw: Any) -> Optional[Tuple[float, bool]]:
76         """(seconds, was_mmss) or None if blank/unparseable. Decimal seconds or mm:ss /
HH:MM:SS."""
77         if raw is None:
78             return None
79         s = str(raw).strip()
80         if not s:
81             return None
82         try:
83             if ":" in s:
84                 parts = [float(p) for p in s.split(":")]
85                 if len(parts) == 3:
86                     return parts[0] * 3600 + parts[1] * 60 + parts[2], True
87                 if len(parts) == 2:
88                     return parts[0] * 60 + parts[1], True
89                 return None
90             return float(s), False
91         except ValueError:
92             return None
93
94
95     def _primary_reason(raw: Any) -> Tuple[str, str]:
96         """Pipe-delimited 'forced_error|other' -> ('forced_error', 'other'). Blank -> ('',
'')."""
97         if raw is None:
98             return "", ""
99         s = str(raw).strip()
100         if not s:
101             return "", ""
102         parts = [p.strip() for p in s.split("|") if p.strip()]
103         if not parts:
104             return "", ""
105         return parts[0], "; ".join(parts[1:])
106
107
108     def _get(row: Dict[str, Any], *keys: str) -> Any:
109         """First non-empty value among the given (already-lowercased) keys."""
110         for k in keys:
111             v = row.get(k)
112             if v is not None and str(v).strip() != "":
113                 return v
114         return None
115
116
117     def normalize_rows(rows: List[Dict[str, Any]]) -> Tuple[List[Dict[str, Any]],
NormalizeReport]:
118         """Pure core: lowercase-keyed annotator rows -> clean lean rows + report."""
119         rep = NormalizeReport(rows_in=len(rows))
120         kept: List[Dict[str, Any]] = []
121
122         for row in rows:
123             orig_id = _get(row, "rally_number", "rally")
124             s_parsed = _to_seconds(_get(row, "start_time", "start_mmss", "start"))
125             e_parsed = _to_seconds(_get(row, "end_time", "end_mmss", "end"))

```

```

126         start = s_parsed[0] if s_parsed else None
127         end = e_parsed[0] if e_parsed else None
128
129         if start is None:
130             rep.dropped.append({"orig_id": orig_id, "reason": "missing/invalid
start_time",
131                                 "start": _get(row, "start_time", "start_mmss"), "end":
_get(row, "end_time", "end_mmss")})
132             continue
133         if end is None:
134             rep.dropped.append({"orig_id": orig_id, "reason": "missing/invalid
end_time",
135                                 "start": start, "end": _get(row, "end_time",
"end_mmss")})
136             continue
137         if end <= start:
138             rep.dropped.append({"orig_id": orig_id, "reason": "end_time <= start_time",
139                                 "start": start, "end": end})
140             continue
141         if (s_parsed and s_parsed[1]):
142             rep.mmss_converted += 1
143         if (e_parsed and e_parsed[1]):
144             rep.mmss_converted += 1
145
146         reason, reason_note = _primary_reason(_get(row, "ending_reason"))
147         note_in = _get(row, "notes", "note", "reviewer_comment")
148         notes = "; ".join([x for x in [reason_note, (str(note_in).strip() if note_in
else "")] if x])
149         kept.append({
150             "orig_id": orig_id, "start": float(start), "end": float(end),
151             "ending_reason": reason, "sport": _get(row, "sport"),
152             "shots_count": _get(row, "shots_count", "shots"), "notes": notes,
153         })
154
155     kept.sort(key=lambda r: r["start"])
156
157     # Renumber 1..N by start order; detect whether ids were fractional / duplicate /
158     # not already a clean 1..N sequence (any of which means import-local would collide).
159     orig_seq = [str(r["orig_id"]) if r["orig_id"] is not None else "" for r in kept]
160     new_seq = [str(i) for i in range(1, len(kept) + 1)]
161     rep.renumbered = orig_seq != new_seq
162     seen: Dict[str, int] = {}
163     for oid in orig_seq:
164         seen[oid] = seen.get(oid, 0) + 1
165     rep.duplicate_ids = sorted({oid for oid, c in seen.items() if c > 1 and oid})
166     rep.fractional_ids = sorted({oid for oid in orig_seq if "." in oid})
167
168     out: List[Dict[str, Any]] = []
169     for i, r in enumerate(kept, start=1):
170         out.append({
171             "rally_number": i,
172             "start_time": f"{r['start']:.3f}",
173             "end_time": f"{r['end']:.3f}",
174             "ending_reason": r["ending_reason"] or "",
175             "sport": r["sport"] or "",
176             "shots_count": "" if r["shots_count"] is None else
str(r["shots_count"]).strip(),
177             "original_rally_number": "" if r["orig_id"] is None else str(r["orig_id"]),
178             "notes": r["notes"],
179         })
180     rep.rows_out = len(out)
181     return out, rep
182
183
184     def normalize_file(in_path: str, out_path: str) -> NormalizeReport:

```

```

185     with open(in_path, "r", newline="", encoding="utf-8-sig") as f:
186         reader = csv.DictReader(f)
187         reader.fieldnames = [(n or "").lower().strip() for n in (reader.fieldnames or
188         [])]
189         rows = [{(k or "").lower().strip(): v for k, v in r.items()} for r in reader]
190     out, rep = normalize_rows(rows)
191     os.makedirs(os.path.dirname(os.path.abspath(out_path)) or ".", exist_ok=True)
192     with open(out_path, "w", newline="", encoding="utf-8") as f:
193         w = csv.DictWriter(f, fieldnames=OUT_FIELDS)
194         w.writeheader()
195         w.writerows(out)
196     return rep
197
198     def main() -> None:
199         ap = argparse.ArgumentParser(description="Normalize a rally-annotator CSV for safe
import-local.")
200         ap.add_argument("--in", dest="in_path", required=True, help="rally-annotator
.rallies.csv")
201         ap.add_argument("--out", dest="out_path", required=True, help="clean lean CSV to
write")
202         args = ap.parse_args()
203         rep = normalize_file(args.in_path, args.out_path)
204         print("=== normalize-annotations ===")
205         print(rep.summary())
206         print(f"wrote -> {args.out_path}")
207
208
209     if __name__ == "__main__":
210         main()
211

```

4. user (2026-06-08T10:22:58.367Z)

```

1     """normalize_annotations: the two-landmine guard for partial human annotator CSVs.
2
3     Pins the behavior that protects import-local: drop unusable rows (instead of aborting
4     the whole import) and renumber fractional/duplicate rally ids (instead of a silent PK
5     collision). The final test feeds the normalized output through the REAL import-local
6     parser to prove it ingests cleanly.
7     """
8     import csv as _csv
9
10    from src.cli.normalize_annotations import normalize_rows, normalize_file
11    from src.cli.curate import SportsDataCLI
12
13
14    def _rows(*dicts):
15        return [dict(d) for d in dicts]
16
17
18    def test_drops_blank_and_reversed_end_rows():
19        out, rep = normalize_rows(_rows(
20            {"rally_number": "1", "start_time": "10.0", "end_time": "15.0"}, # good
21            {"rally_number": "2", "start_time": "20.0", "end_time": ""}, # blank end
-> drop
22            {"rally_number": "3", "start_time": "30.0", "end_time": "28.0"}, # end<=start
-> drop
23            {"rally_number": "4", "start_time": "40.0", "end_time": "45.0"}, # good
24        ))
25        assert rep.rows_in == 4 and rep.rows_out == 2
26        assert len(rep.dropped) == 2
27        assert [r["start_time"] for r in out] == ["10.000", "40.000"]
28        # surviving rows are cleanly renumbered 1..2
29        assert [r["rally_number"] for r in out] == [1, 2]

```

```

30
31
32 def test_renumbers_fractional_and_duplicate_ids_by_start():
33     out, rep = normalize_rows(_rows(
34         {"rally_number": "7", "start_time": "100.0", "end_time": "105.0"},
35         {"rally_number": "7.5", "start_time": "106.0", "end_time": "110.0"}, # inserted
correction
36         {"rally_number": "8", "start_time": "120.0", "end_time": "125.0"},
37     ))
38     assert rep.renumbered is True
39     assert rep.fractional_ids == ["7.5"]
40     # contiguous integer PKs in start order -> no (video_id, rally_number) collision
41     assert [r["rally_number"] for r in out] == [1, 2, 3]
42     assert [r["original_rally_number"] for r in out] == ["7", "7.5", "8"]
43
44
45 def test_duplicate_truncating_ids_are_separated():
46     # Two ids that BOTH int(float()) to 7 -> would collide on PK; must become 1,2.
47     out, rep = normalize_rows(_rows(
48         {"rally_number": "7.1", "start_time": "10.0", "end_time": "12.0"},
49         {"rally_number": "7.9", "start_time": "20.0", "end_time": "22.0"},
50     ))
51     assert [r["rally_number"] for r in out] == [1, 2]
52     assert rep.renumbered is True
53
54
55 def test_mmss_times_converted_and_counted():
56     out, rep = normalize_rows(_rows(
57         {"rally": "1", "start_mmss": "1:46.031", "end_mmss": "1:50.000"},
58     ))
59     assert out[0]["start_time"] == "106.031" and out[0]["end_time"] == "110.000"
60     assert rep.mmss_converted == 2
61
62
63 def test_pipe_delimited_reason_folds_to_primary_plus_note():
64     out, _ = normalize_rows(_rows(
65         {"rally_number": "1", "start_time": "1.0", "end_time": "2.0", "ending_reason":
"forced_error|other"},
66     ))
67     assert out[0]["ending_reason"] == "forced_error"
68     assert "other" in out[0]["notes"]
69
70
71 def test_clean_lean_input_is_passthrough():
72     out, rep = normalize_rows(_rows(
73         {"rally_number": "1", "start_time": "1.0", "end_time": "2.0", "ending_reason":
"winner", "sport": "badminton"},
74         {"rally_number": "2", "start_time": "3.0", "end_time": "5.0", "ending_reason":
"unforced_error", "sport": "badminton"},
75     ))
76     assert rep.rows_out == 2 and rep.renumbered is False and not rep.dropped
77     assert [r["sport"] for r in out] == ["badminton", "badminton"]
78
79
80 def test_normalized_output_ingests_through_import_local_parser(tmp_path):
81     """End-to-end: a dirty annotator CSV -> normalize -> the REAL parser yields N
82     BadmintonRally with contiguous int ids and no exception (the import-abort + PK
83     collision are both gone)."""
84     dirty = tmp_path / "dirty.rallies.csv"
85     with open(dirty, "w", newline="", encoding="utf-8") as f:
86         w = _csv.writer(f)
87         w.writerow(["rally_number", "start_time", "end_time", "ending_reason", "sport"])
88         w.writerow([1, 37.434, 72.194, "winner", "badminton"])
89         w.writerow([2, 79.936, "", "winner", "badminton"]) # blank end (would
abort)

```

```

90         w.writerow([2.5, 90.0, 95.0, "unforced_error", "badminton"]) # fractional
(would collide)
91         w.writerow([3, 130.193, 134.947, "winner", "badminton"])
92         clean = tmp_path / "clean.csv"
93         rep = normalize_file(str(dirty), str(clean))
94         assert rep.rows_out == 3 and len(rep.dropped) == 1
95
96         # self is unused inside _parse_local_annotations -> None is safe.
97         items = SportsDataCLI._parse_local_annotations(None, "badminton", "adarsh_avi",
str(clean))
98         assert len(items) == 3
99         assert [it.rally_number for it in items] == [1, 2, 3] # contiguous, no collision
100         assert all(it.end_time > it.start_time for it in items)
101

```

5. user (2026-06-08T10:22:58.827Z)

```

1      # Positioning: manual (rally-annotator) golden-label ingestion
2
3      How `sports-data-collector` should be positioned to ingest manual rally labels
4      produced by the VLC [rally-annotator](https://github.com/avidullu/rally-annotator),
5      alongside vendor (Roora/CVAT) deliveries, and serve them to the
6      badminton-highlight-indexer eval/training pipeline.
7
8      > Source: a 10-agent analysis workflow across this repo, the indexer, and the
9      > annotator, verified against the real `Adarsh and Avi.rallies.csv`. Confidence: high.
10
11     ## Bottom line
12
13     Position the collector as the single canonical system-of-record for golden rally
14     labels: the one place where any annotation source ? manual VLC lean CSV, the rich
15     review sheet, or vendor CVAT/CSV ? becomes a validated, license-gated, registered rally
16     record. The indexer consumes only the collector's narrow export contract:
17     `manifest.json` + a per-video `start,end` CSV (`curate.py::export_eval_set`;
18     `calibrate_wasb.py::load_golden_gts` reads `start_time`/`end_time` only).
19
20     The load-bearing property to protect: the seam to the indexer stays intervals-only.
21     Everything else ? `ending_reason`, `shots_count`, scores, provenance, maturity, the
22     labeled window ? is validated at the hub and rides in the manifest, never the eval
23     CSV. A new sport or source becomes an adapter change at the hub, never an indexer
24     change.
25
26     ## Role in the pipeline
27
28     ``
29     producers ?????????????? collector (hub) ?????????????? consumer
30     ? VLC rally-annotator      parse ? validate ? license-gate  badminton-highlight-
31     (lean 5-col CSV)           ? register (SQLite) ? export      indexer (eval/training)
32     ? manual review sheet      everything but (start,end) is    reads ONLY:
33     (rich 14-col CSV)          validated then dropped before    manifest.json +
34     ? vendor Roora / CVAT      per-video start,end             per-video start,end
35
36     CSV
37     (CVAT XML / canonical)     the indexer sees it
38     ``
39
40     ## It already works today (zero code changes)
41
42     The lean annotator CSV header `rally_number,start_time,end_time,ending_reason,sport`
43     matches the `import-local` `DictReader` exactly (`curate.py::_parse_local_annotations`),
44     and the validator accepts gapped/partial coverage as-is
45     (`validator.py::validate_badminton_rallies` checks only positive duration + no overlap).
46     Verified end-to-end on the real file ? 40 rallies ? export ? 40-interval indexer CSV.
47
48     ``bash
49     python -m src.cli.curate import-local --sport badminton \

```

```

47     --video-path "?/Adarsh and Avi.MP4" \
48     --annotations-path "?/Adarsh and Avi.rallies.csv" --license Proprietary --fps 59.94
49 python -m src.cli.curate export # ? manifest.json + <video_id>.csv (start,end)
50 ```
51
52 ## P0 ? the two-landmine guard (`normalize_annotations.py`, shipped)
53
54 Partial human data has two failure modes the raw path handles badly. The
55 `src/cli/normalize_annotations.py` pre-import normalizer fixes both, as a **standalone
56 step** that feeds `import-local` (it does **not** touch the DB schema or the
57 `rally_number`-ordered export):
58
59 1. **A blank or `end_time <= start_time` row aborts the *entire* import.**
60   `safe_float_required` raises (`parsing.py`), killing the whole video. ? the
61   normalizer **drops** such rows (mirroring the indexer loader, which already skips
62   `not end > start`).
63 2. **A fractional/duplicate `rally_number` (e.g. `7.5`, `16.5`) silently corrupts
data.**
64   `int(float("7.5"))` ? 7` (`parsing.py::safe_int`) collides on the
65   `(video_id, rally_number)` primary key, and the DELETE-then-INSERT upsert
66   (`db.py::insert_badminton_rallies`) destroys the colliding rally. ? the normalizer
67   **renumbers `1..N` by `start_time`**, recording the original id for traceability.
68
69 It also converts `mm:ss`/`HH:MM:SS` ? decimal seconds and folds a pipe-delimited
70 `ending_reason` ("forced_error|other") to its primary token.
71
72 ```bash
73 python -m src.cli.normalize_annotations --in "Match.rallies.csv" --out clean.csv
74 python -m src.cli.curate import-local --sport badminton --video-path "?/Match.MP4" \
75 --annotations-path clean.csv --license Proprietary
76 ```
77
78 **Do not widen the PK to float** ? the DB column is INTEGER, export orders by it, and
79 the indexer ignores `rally_number` entirely. Resolve ids at the adapter boundary.
80
81 ## Annotator ? canonical field mapping
82
83 | Annotator field | Canonical | Handling |
84 |---|---|---|
85 | `rally_number` (lean) / `rally` (rich) | `rally_number:int` | Lean maps directly;
**fractional/dup ids renumbered at the normalizer** (PK is INT) |
86 | `start_time` / `start_mmss` | `start_time:float` | Decimal direct; `mm:ss` converted
to seconds |
87 | `end_time` / `end_mmss` | `end_time:float` | **Blank / ? start dropped, not raised** |
88 | `ending_reason` | `ending_reason:str` | Vocab matches canonical enum; pipe-delimited
folded to primary; **stored but dropped on export** |
89 | `sport` | ? (from `--sport` flag) | CSV column ignored; operator sets the flag |
90 | rich-only `shots`, `true_*`, `VERIFIED`, `notes` | `shots_count` + QA/provenance |
Carried where present; none reach the indexer |
91
92 ## Roadmap to the canonical-hub end-state
93
94 All additive, video-level, backward-compatible (consumers ignore unknown manifest keys).
95
96 - **P1 ? persist the labeled window in the manifest.** Partial labels cover only a
97 prefix; a full-video eval penalizes correct detections in the unlabeled tail. The
98 indexer already restricts to a window (`eval_partial_labels.py::restrict_to_window`)
99 but currently *infers* `window_end = max(end_time)` ? which is **wrong when a video is
100 tagged past its last rally** (valid predictions in the labeled-but-rally-free tail get
101 scored as false positives). The collector should own `labeled_window_start/end`
102 (the VLC tool is resumable + monotonic, so the session extent is known), carry it in
103 the manifest, and let the indexer apply it automatically. Short-term: pass
104 `--window-end` at eval time.
105 - **P1 ? maturity ladder + provenance.** `CurationStatus` (STAGING/CURATED/REJECTED) is
106 a *workflow* state, not a *trust* state. Add a separate `needs_review` ? confirmed ?

```



```

107 gold` enum + `annotation_source` (manual_vlc / manual_review / vendor_cvat) +
108 `annotator_id` + timestamp. Route self-rated imports to `needs_review` instead of the
109 hardcoded `CURATED` (`curate.py`), so the golden-set regression flow restricts the
gold
110 tier to `maturity=gold` and an unreviewed re-tag can't silently become the answer key.
111 (Today a single-rater self-rating and a vendor-QA'd delivery are indistinguishable.)
112 - **P2 ? unify sources behind one adapter** emitting `(List[RallyRecord], List[Issue])`,
113 and run the **same Issue detectors** (gap>15s possible-missed-rally, shot-density,
114 vocab, overlap) on manual imports that vendor CVAT already gets. Today self-produced
115 labels get *less* scrutiny than a vendor delivery ? backwards.
116 - **P2 ? freeze the intervals-only export contract** in one doc both repos reference +
117 a round-trip test (header matching is exact case-insensitive, so a vendor
118 `start_seconds` would fail silently).
119
120 ## Open question
121
122 Is the **rich** `rally_review.csv` (mm:ss times, a `rally` column with fractional ids,
123 pipe-delimited reasons, `true_` correction columns) ever the *source of truth*, or is
124 the lean VLC `rallies.csv` always canonical? Neither `import-local` nor
125 `load_canonical_csv` (which keys on `rally_number`) ingest the rich file directly. The
126 `normalize_annotations.py` step already handles its `mm:ss` + `rally` + pipe-reason
127 quirks for the common case; a fuller converter that applies `true_` overrides is only
128 worth building if the rich sheet is canonical.
129

```

6. assistant (2026-06-08T10:23:01.745Z)

Now let me read the integration points: `curate.py`, `parsing.py`, `db.py`, and the indexer eval file.

7. user (2026-06-08T10:23:02.779Z)

```

220         elif sport == "tabletennis":
221             parsed_items.append(TableTennisRally(
222                 video_id=video_id,
223                 rally_number=_safe_int(obj.get("rally_number"), idx + 1),
224                 start_time=_safe_float(obj.get("start_time")),
225                 end_time=_safe_float(obj.get("end_time")),
226                 score_before=obj.get("score_before"),
227                 score_after=obj.get("score_after"),
228                 shots_count=_safe_int(obj.get("shots_count"), None),
229                 ending_reason=obj.get("ending_reason"),
230                 last_shot_outcome=obj.get("last_shot_outcome")
231             ))
232         elif ext == '.csv':
233             with open(filepath, 'r', encoding='utf-8') as f:
234                 reader = csv.DictReader(f)
235                 # Normalize headers
236                 reader.fieldnames = [n.lower().strip() for n in reader.fieldnames] if
reader.fieldnames else []
237
238             for idx, row in enumerate(reader):
239                 if sport == "badminton":
240                     parsed_items.append(BadmintonRally(
241                         video_id=video_id,
242                         rally_number=_safe_int(row.get("rally_number"), idx + 1),
243                         start_time=_safe_float(row.get("start_time")),
244                         end_time=_safe_float(row.get("end_time")),
245                         score_before=row.get("score_before"),
246                         score_after=row.get("score_after"),
247                         shots_count=_safe_int(row.get("shots_count"), None),
248                         ending_reason=row.get("ending_reason"),
249                         last_shot_outcome=row.get("last_shot_outcome")
250                     ))
251                 elif sport == "tennis":

```

```

252         parsed_items.append(TennisPoint(
253             video_id=video_id,
254             point_number=_safe_int(row.get("point_number"), idx + 1),
255             start_time=_safe_float(row.get("start_time")),
256             end_time=_safe_float(row.get("end_time")),
257             set_score=row.get("set_score"),
258             game_score=row.get("game_score"),
259             server=row.get("server"),
260             ending_type=row.get("ending_type")
261         ))
262     elif sport == "cricket":
263         parsed_items.append(CricketDelivery(
264             video_id=video_id,
265             over=_safe_int(row.get("over"), 0),
266             ball=_safe_int(row.get("ball"), 0),
267             start_time=_safe_float(row.get("start_time")),
268             end_time=_safe_float(row.get("end_time")),
269             batsman=row.get("batsman"),
270             bowler=row.get("bowler"),
271             runs=_safe_int(row.get("runs"), None),
272             wicket=(row.get("wicket") or "").lower() in ["true", "1",
"yes"],
273             extras=_safe_int(row.get("extras"), 0)
274         ))
275     elif sport == "pickleball":
276         parsed_items.append(PickleballRally(
277             video_id=video_id,
278             rally_number=_safe_int(row.get("rally_number"), idx + 1),
279             start_time=_safe_float(row.get("start_time")),
280             end_time=_safe_float(row.get("end_time")),
281             score_before=row.get("score_before"),
282             score_after=row.get("score_after"),
283             shots_count=_safe_int(row.get("shots_count"), None),
284             ending_reason=row.get("ending_reason"),
285             last_shot_outcome=row.get("last_shot_outcome")
286         ))
287     elif sport == "tabletennis":
288         parsed_items.append(TableTennisRally(
289             video_id=video_id,
290             rally_number=_safe_int(row.get("rally_number"), idx + 1),
291             start_time=_safe_float(row.get("start_time")),
292             end_time=_safe_float(row.get("end_time")),
293             score_before=row.get("score_before"),
294             score_after=row.get("score_after"),
295             shots_count=_safe_int(row.get("shots_count"), None),
296             ending_reason=row.get("ending_reason"),
297             last_shot_outcome=row.get("last_shot_outcome")
298         ))
299     else:
300         raise ValueError("Unsupported annotations file extension. Use .json or
.csv.")
301
302     return parsed_items
303
304 def _fetch_one_with_retry(self, url, dest_path, args, max_height, max_size_mb):
305     """Download one video at best resolution, retrying transient failures.
306
307     A below-min-height result almost always means YouTube rate-limited the
308     request and served a low fallback (not that the video lacks HD), so we
309     retry with a growing backoff. Returns (DownloadResult, None) on success,
310     or (None, failure_reason) once retries are exhausted. A below-min file is
311     discarded each attempt so it never lands in the DB.
312     """
313     attempts = max(1, args.retries + 1)
314     reason = "unknown error"

```

```

315         for i in range(attempts):
316             result = download_best_video(
317                 url, dest_path,
318                 max_height=max_height, max_size_mb=max_size_mb,
319                 cookies_from_browser=args.cookies_from_browser,

```

8. user (2026-06-08T10:23:03.124Z)

```

1     """Shared parsing helpers used by the CLI importer and dataset parsers.
2
3     Previously these were duplicated as private ``_safe_int`` / ``_safe_float`` /
4     ``_parse_time_string`` in both ``src/cli/curate.py`` and the CoachAI parser.
5     """
6     from typing import Any, Optional
7
8
9     def safe_int(value: Any, default: Optional[int] = 0) -> Optional[int]:
10         """Parse an int from a possibly-empty/float/None value; return default on
failure."""
11         if value is None:
12             return default
13         s = str(value).strip()
14         if not s:
15             return default
16         try:
17             return int(float(s))
18         except (TypeError, ValueError):
19             return default
20
21
22     def safe_float_required(value: Any) -> float:
23         """Parse a float; raise ValueError if missing/empty (caller treats it as
required)."""
24         if value is None:
25             raise ValueError("missing required numeric value")
26         s = str(value).strip()
27         if not s:
28             raise ValueError("missing required numeric value")
29         return float(s)
30
31
32     def parse_time_string(time_str: Any) -> float:
33         """Convert 'HH:MM:SS' / 'MM:SS' / float-strings to float seconds; 0.0 if
blank/invalid."""
34         if not time_str:
35             return 0.0
36         s = str(time_str).strip()
37         if ":" in s:
38             parts = s.split(":")
39             try:
40                 if len(parts) == 3:
41                     return float(parts[0]) * 3600 + float(parts[1]) * 60 + float(parts[2])
42                 if len(parts) == 2:
43                     return float(parts[0]) * 60 + float(parts[1])
44             except ValueError:
45                 return 0.0
46         try:
47             return float(s)
48         except ValueError:
49             return 0.0
50

```

9. assistant (2026-06-08T10:23:07.869Z)

Now let me look at the `\_safe\_int` and `\_safe\_float` references in curate.py (they have

underscore prefixes), and read the broader context of `\_parse\_local\_annotations` including the CSV branch start and how `import-local` sets status. Let me also check `db.py::insert\_badminton\_rallies` and the indexer eval file.

10. user (2026-06-08T10:23:09.407Z)

```
11:from src.core.parsing import safe_int as _safe_int, safe_float_required as _safe_float
175:         rally_number=_safe_int(obj.get("rally_number"), idx + 1),
176:         start_time=_safe_float(obj.get("start_time")),
177:         end_time=_safe_float(obj.get("end_time")),
180:         shots_count=_safe_int(obj.get("shots_count"), None),
187:         point_number=_safe_int(obj.get("point_number"), idx + 1),
188:         start_time=_safe_float(obj.get("start_time")),
189:         end_time=_safe_float(obj.get("end_time")),
198:         over=_safe_int(obj.get("over"), 0),
199:         ball=_safe_int(obj.get("ball"), 0),
200:         start_time=_safe_float(obj.get("start_time")),
201:         end_time=_safe_float(obj.get("end_time")),
204:         runs=_safe_int(obj.get("runs"), None),
206:         extras=_safe_int(obj.get("extras"), 0)
211:         rally_number=_safe_int(obj.get("rally_number"), idx + 1),
212:         start_time=_safe_float(obj.get("start_time")),
213:         end_time=_safe_float(obj.get("end_time")),
216:         shots_count=_safe_int(obj.get("shots_count"), None),
223:         rally_number=_safe_int(obj.get("rally_number"), idx + 1),
224:         start_time=_safe_float(obj.get("start_time")),
225:         end_time=_safe_float(obj.get("end_time")),
228:         shots_count=_safe_int(obj.get("shots_count"), None),
242:         rally_number=_safe_int(row.get("rally_number"), idx + 1),
243:         start_time=_safe_float(row.get("start_time")),
244:         end_time=_safe_float(row.get("end_time")),
247:         shots_count=_safe_int(row.get("shots_count"), None),
254:         point_number=_safe_int(row.get("point_number"), idx + 1),
255:         start_time=_safe_float(row.get("start_time")),
256:         end_time=_safe_float(row.get("end_time")),
265:         over=_safe_int(row.get("over"), 0),
266:         ball=_safe_int(row.get("ball"), 0),
267:         start_time=_safe_float(row.get("start_time")),
268:         end_time=_safe_float(row.get("end_time")),
271:         runs=_safe_int(row.get("runs"), None),
273:         extras=_safe_int(row.get("extras"), 0)
278:         rally_number=_safe_int(row.get("rally_number"), idx + 1),
279:         start_time=_safe_float(row.get("start_time")),
280:         end_time=_safe_float(row.get("end_time")),
283:         shots_count=_safe_int(row.get("shots_count"), None),
290:         rally_number=_safe_int(row.get("rally_number"), idx + 1),
291:         start_time=_safe_float(row.get("start_time")),
292:         end_time=_safe_float(row.get("end_time")),
295:         shots_count=_safe_int(row.get("shots_count"), None),
```

11. user (2026-06-08T10:23:10.367Z)

```
180         shots_count=_safe_int(obj.get("shots_count"), None),
181         ending_reason=obj.get("ending_reason"),
182         last_shot_outcome=obj.get("last_shot_outcome")
183     ))
184     elif sport == "tennis":
185         parsed_items.append(TennisPoint(
186             video_id=video_id,
187             point_number=_safe_int(obj.get("point_number"), idx + 1),
188             start_time=_safe_float(obj.get("start_time")),
189             end_time=_safe_float(obj.get("end_time")),
190             set_score=obj.get("set_score"),
191             game_score=obj.get("game_score"),
192             server=obj.get("server"),
```

```

193         ending_type=obj.get("ending_type")
194     ))
195     elif sport == "cricket":
196         parsed_items.append(CricketDelivery(
197             video_id=video_id,
198             over=_safe_int(obj.get("over"), 0),
199             ball=_safe_int(obj.get("ball"), 0),
200             start_time=_safe_float(obj.get("start_time")),
201             end_time=_safe_float(obj.get("end_time")),
202             batsman=obj.get("batsman"),
203             bowler=obj.get("bowler"),
204             runs=_safe_int(obj.get("runs"), None),
205             wicket=bool(obj.get("wicket", False)),
206             extras=_safe_int(obj.get("extras"), 0)
207         ))
208     elif sport == "pickleball":
209         parsed_items.append(PickleballRally(
210             video_id=video_id,
211             rally_number=_safe_int(obj.get("rally_number"), idx + 1),
212             start_time=_safe_float(obj.get("start_time")),
213             end_time=_safe_float(obj.get("end_time")),
214             score_before=obj.get("score_before"),
215             score_after=obj.get("score_after"),
216             shots_count=_safe_int(obj.get("shots_count"), None),
217             ending_reason=obj.get("ending_reason"),
218             last_shot_outcome=obj.get("last_shot_outcome")
219         ))
220     elif sport == "tabletennis":
221         parsed_items.append(TableTennisRally(
222             video_id=video_id,
223             rally_number=_safe_int(obj.get("rally_number"), idx + 1),
224             start_time=_safe_float(obj.get("start_time")),

```

12. user (2026-06-08T10:23:10.699Z)

```

1     """Evaluate the rally pipeline against PARTIAL human golden labels.
2
3     The VLC rally-annotator produces a human-rated CSV (rally_number,start_time,end_time,
4     ending_reason,sport). While a video is only partially tagged, the labels cover a
5     PREFIX of the video ? every rally up to some point is marked, and everything after is
6     simply un-watched (not "no rally there"). Scoring naively against such labels is wrong:
7     the detector's *correct* rallies in the un-labeled tail would be counted as false
8     positives and crush precision.
9
10    This harness fixes that by restricting the evaluation to the LABELED WINDOW
11    [window_start, window_end] (window_end defaults to the last labeled rally's end) and
12    dropping predicted segments that fall outside it. Within that window the human labels
13    ARE complete, so precision/recall/F1 are honest.
14
15    It reuses everything from the existing eval stack (no new windowing/metrics/gate logic):
16    - calibrate_wasb.make_predict_fn / load_golden_gts / BASELINE
17    - pipeline.detectors.rally_gate.apply_gate (net-crossing "gate A")
18    - eval.metrics.match_intervals / score
19    - export_wasb_segments.build_comparison / write_comparison_csv (per-rally detail CSV)
20
21    Run:
22    python -m backend.eval.eval_partial_labels \
23        --trajectory C:/Users/avidu/AppData/Local/Temp/adarsh_avi_traj.csv \
24        --labels "C:/Users/avidu/Projects/Annotation Setup/Collect/Adarsh and
25    Avi.rallies.csv" \
26        --fps 59.94 --frame-width 1920
27
28    """
29    from __future__ import annotations
30
31    import argparse

```

```

30 import os.path as osp
31 from typing import List, Optional, Tuple
32
33 from backend.eval.calibrate_wasb import make_predict_fn, load_golden_gts, BASELINE
34 from backend.eval.metrics import score
35 from backend.eval.export_wasb_segments import build_comparison, write_comparison_csv
36 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
37 from backend.pipeline.detectors import rally_gate
38
39 Interval = Tuple[float, float]
40 Config = Tuple[float, float, float]
41
42 # Operating points to compare. The tuned config wins on its fit-video but over-segments
43 # and does not transfer; baseline (+ net-crossing gate) is the stronger general point.
44 TUNED: Config = (0.05, 1.0, 1.0)
45
46
47 def restrict_to_window(intervals: List[Interval], w0: float, w1: float) ->
List[Interval]:
48     """Keep intervals that overlap the labeled window [w0, w1].
49
50     A predicted segment is evaluated iff it overlaps the labeled region; segments wholly
51     in the un-labeled tail (start >= w1) or before the window (end <= w0) are dropped so
52     they are neither rewarded nor penalized. GTs are assumed already inside the window.
53     """
54     return [(s, e) for (s, e) in intervals if e > w0 and s < w1]
55
56
57 def _eval_point(predict_fn, cfg: Config, points, fps: float, gts: List[Interval],
58                 window: Tuple[float, float], gate_crossings: int, iou: float) -> dict:
59     """Score one (config, gate) operating point inside the labeled window."""
60     preds = predict_fn(cfg)
61     if gate_crossings > 0 and points:
62         preds = rally_gate.apply_gate(points, preds, fps, min_crossings=gate_crossings)
63     preds = restrict_to_window(preds, *window)
64     sc = score(preds, gts, iou)
65     return {"cfg": cfg, "gate": gate_crossings, "n_pred": len(preds), "score": sc,
66 "preds": preds}
67
68
69 def run(trajjectory_csv: str, labels_csv: str, fps: float, frame_width: float,
70        window_start: float = 0.0, window_end: Optional[float] = None,
71        gate_crossings: int = 2, iou: float = 0.5,
72        comparison_out: Optional[str] = None) -> dict:
73     gts_all = load_golden_gts(labels_csv)
74     if not gts_all:
75         raise SystemExit(f"no usable human rallies in {labels_csv}")
76     if window_end is None:
77         window_end = max(e for _, e in gts_all)
78     window = (window_start, window_end)
79     gts = restrict_to_window(gts_all, *window)
80
81     predict_fn = make_predict_fn(trajjectory_csv, fps, frame_width)
82     points = TrackNetRunner.parse_trajectory_csv(trajjectory_csv)
83     traj_end = (max(p.frame for p in points) / fps) if points else 0.0
84
85     print("=== Pipeline vs PARTIAL human labels ===")
86     print(f"labels: {labels_csv}")
87     print(f" {len(gts_all)} human rallies; labeled window [{window_start:.1f}s,
88 {window_end:.1f}s] "
89           f"-> {len(gts)} rallies in-window")
90     print(f"trajectory covers ~0-{traj_end:.1f}s "
91           f"({'covers the window' if traj_end >= window_end - 1 else 'WARNING: shorter
92 than window'})")
93     print(f"IoU>={iou}; net-crossing gate K={gate_crossings}\n")

```

```

91
92     # Compare the standard operating points: baseline / baseline+gate / tuned /
tuned+gate.
93     points_for_gate = points
94     candidates = [
95         ("baseline", BASELINE, 0),
96         (f"baseline+gate{gate_crossings}", BASELINE, gate_crossings),
97         ("tuned", TUNED, 0),
98         (f"tuned+gate{gate_crossings}", TUNED, gate_crossings),
99     ]
100     results = []
101     header = f"{'operating point':<18} {'pred':>4} {'TP':>3} {'FN':>3} {'FP':>3} " \
102             f"{'prec':>6} {'recall':>6} {'F1':>6} {'mIoU':>6} {'sErr':>6} {'eErr':>6}"
103     print(header)
104     print("-" * len(header))
105     for name, cfg, gate in candidates:
106         r = _eval_point(predict_fn, cfg, points_for_gate, fps, gts, window, gate, iou)
107         sc = r["score"]
108         r["name"] = name
109         results.append(r)
110         print(f"{name:<18} {r['n_pred']:>4} {sc.true_positives:>3}
{sc.false_negatives:>3} "
111             f"{sc.false_positives:>3} {sc.precision:>6.3f} {sc.recall:>6.3f}
{sc.f1:>6.3f} "
112             f"{sc.mean_iou:>6.3f} {sc.mean_start_error:>6.2f}
{sc.mean_end_error:>6.2f}")
113
114     best = max(results, key=lambda r: r["score"].f1)
115     print(f"\nbest operating point by F1: {best['name']} (F1 {best['score'].f1:.3f})")
116
117     # Per-rally detail for the best point: which human rallies matched/missed + extra
fires.
118     comp = build_comparison(best["preds"], gts, iou)
119     comp_out = comparison_out or osp.join("output", "adarsh_avi_vs_human.csv")
120     write_comparison_csv(comp_out, comp)
121     print(f"per-rally detail ({best['name']}) -> {comp_out}")
122
123     return {
124         "window": window, "n_human_in_window": len(gts),
125         "results": [{ "name": r["name"], "cfg": r["cfg"], "gate": r["gate"],
126                     "n_pred": r["n_pred"], **r["score"].as_dict() } for r in results],
127         "best": best["name"], "comparison_csv": comp_out,
128     }
129
130
131     # Grid centered on the two levers the human-GT failure analysis flagged:
132     # min_window_duration DOWN (recover short 3-5s rallies) and merge_gap UP (heal
133     # fragmented long rallies). Gate is swept on/off because it can over-prune.
134     DEFAULT_GRID = [
135         (vt, mg, mwd)
136         for vt in (0.01, 0.02, 0.03, 0.05)
137         for mg in (1.0, 2.0, 3.0, 4.0)
138         for mwd in (0.5, 1.0, 1.5, 2.0)
139     ]
140
141
142     def sweep(trajjectory_csv: str, labels_csv: str, fps: float, frame_width: float,
143             window_start: float = 0.0, window_end: Optional[float] = None,
144             gate_options: Tuple[int, ...] = (0, 2), iou: float = 0.5,
145             grid: Optional[List[Config]] = None, top: int = 12) -> dict:
146         """Grid-search (velocity, merge_gap, min_dur) x gate vs human labels, in-window.
147
148         WARNING: this is fit-on-self on ONE video -> optimistic. It shows the achievable
149         ceiling + which levers matter, NOT a transferable config. The honest operating
150         point comes from leave-one-video-out once >=3 videos are labeled.

```

```

151     """
152     grid = grid or DEFAULT_GRID
153     gts_all = load_golden_gts(labels_csv)
154     if not gts_all:
155         raise SystemExit(f"no usable human rallies in {labels_csv}")
156     if window_end is None:
157         window_end = max(e for _, e in gts_all)
158     window = (window_start, window_end)
159     gts = restrict_to_window(gts_all, *window)
160     predict_fn = make_predict_fn(trajecory_csv, fps, frame_width)
161     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
162
163     rows: List[dict] = []
164     for cfg in grid:
165         for gate in gate_options:
166             r = _eval_point(predict_fn, cfg, points, fps, gts, window, gate, iou)
167             sc = r["score"]
168             rows.append({"cfg": cfg, "gate": gate, "n_pred": r["n_pred"], "f1": sc.f1,
169                         "precision": sc.precision, "recall": sc.recall, "mean_iou":
170                         "tp": sc.true_positives, "fp": sc.false_positives, "fn":
171                         sc.false_negatives})
172
173     rows.sort(key=lambda x: (x["f1"], x["mean_iou"]), reverse=True)
174
175     print(f"=== Config sweep vs human labels ({len(gts)} rallies in "
176           f"{window_start:.0f},{window_end:.0f}]s; {len(grid)}x{len(gate_options)}
177           configs) ===")
178     print("NOTE: fit-on-self on ONE video = optimistic; real test is leave-one-video-
179           out.\n")
180     hdr = f"{'#':>3} {'veloc':>6} {'merge':>6} {'minDur':>6} {'gate':>4} " \
181           f"{'F1':>6} {'prec':>6} {'recall':>6} {'mIoU':>6} {'TP':>3} {'FP':>3}
182           {'FN':>3}"
183     print(hdr); print("-" * len(hdr))
184     for i, r in enumerate(rows[:top], 1):
185         vt, mg, mwd = r["cfg"]
186         print(f"{i:>3} {vt:>6.3f} {mg:>6.1f} {mwd:>6.1f} {r['gate']:>4} "
187               f"{r['f1']:>6.3f} {r['precision']:>6.3f} {r['recall']:>6.3f}
188               {r['mean_iou']:>6.3f} "
189               f"{r['tp']:>3} {r['fp']:>3} {r['fn']:>3}")
190     b = rows[0]
191     print(f"\nbest: velocity={b['cfg'][0]} merge_gap={b['cfg'][1]} min_dur={b['cfg'][2]}
192           "
193           f"gate={b['gate']} -> F1 {b['f1']:.3f} (P {b['precision']:.3f} / R
194           {b['recall']:.3f})")
195     return {"window": window, "n_gt": len(gts), "ranked": rows, "best": b}
196
197 def main() -> None:
198     ap = argparse.ArgumentParser(description=__doc__)
199     ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
200 (Frame,Visibility,X,Y)")
201     ap.add_argument("--labels", required=True, help="human golden CSV
202 (rally_number,start_time,end_time,...)")
203     ap.add_argument("--fps", type=float, default=59.94)
204     ap.add_argument("--frame-width", type=float, default=1920.0)
205     ap.add_argument("--window-start", type=float, default=0.0)
206     ap.add_argument("--window-end", type=float, default=None,
207                     help="end of the labeled region (default: last labeled rally end)")
208     ap.add_argument("--gate", type=int, default=2, help="net-crossing gate K (0=off, 2
209 recommended)")
210     ap.add_argument("--iou", type=float, default=0.5)
211     ap.add_argument("--comparison-out", default=None)
212     ap.add_argument("--sweep", action="store_true",
213                     help="grid-search configs vs labels (fit-on-self; finds the ceiling
214 + best levers)")

```



```

204     ap.add_argument("--top", type=int, default=12, help="how many sweep rows to print")
205     args = ap.parse_args()
206     if args.sweep:
207         sweep(args.trajectory, args.labels, args.fps, args.frame_width,
208               window_start=args.window_start, window_end=args.window_end,
209               iou=args.iou, top=args.top)
210     else:
211         run(args.trajectory, args.labels, args.fps, args.frame_width,
212             window_start=args.window_start, window_end=args.window_end,
213             gate_crossings=args.gate, iou=args.iou, comparison_out=args.comparison_out)
214
215
216 if __name__ == "__main__":
217     main()
218

```

13. assistant (2026-06-08T10:23:13.915Z)

Now I have the key integration points. Let me verify a few specifics: the doc's claims about `load\_golden\_gts`, `export\_eval\_set`, `CurationStatus`, and the hardcoded `CURATED` in import-local. Let me also confirm how `import-local` sets status and how the export orders/exports.

14. user (2026-06-08T10:23:15.330Z)

```

1     """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
2     "+X%".
3
4     Wires the already-tested pieces together:
5     WASB trajectory CSV (Frame,Visibility,X,Y)
6     -> TrackNetRunner.trajectory_to_action_windows(cfg)    [predict_fn]
7     -> backend.eval.calibration (improvement_report / cross_validate)
8
9     Run (after a full-video WASB pass exists):
10    python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
11        --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
12
13    Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
14    same GT is an **optimistic** upper bound. The trustworthy number is the
15    cross-validated held-out **recall** (and, once a 2nd labelled video exists,
16    `leave_one_video_out` for precision/F1 generalization).
17    """
18
19    from __future__ import annotations
20
21    import csv
22    import argparse
23    from typing import List, Tuple, Callable
24
25    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
26    from backend.eval.calibration import (
27        select_best, improvement_report, cross_validate, evaluate,
28    )
29
30    Interval = Tuple[float, float]
31    Config = Tuple[float, float, float]    # (velocity_thresh, merge_gap,
32    min_window_duration)
33
34    BASELINE: Config = (0.02, 2.0, 2.0)    # the WasbConfig / trajectory_to_action_windows
35    defaults
36
37
38    def load_golden_gts(csv_path: str) -> List[Interval]:
39        """Rally [start,end] seconds from the golden CSV (start_time/end_time cols)."""
40        gts: List[Interval] = []
41        with open(csv_path, newline="", encoding="utf-8") as f:
42            reader = csv.DictReader(f)

```

```

39         reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
[[]])]
40     for row in reader:
41         row = {(k or "").strip().lower(): v for k, v in row.items()}
42         try:
43             s = float((row.get("start_time") or "").strip())
44             e = float((row.get("end_time") or "").strip())
45         except (TypeError, ValueError):
46             continue
47         if e > s:
48             gts.append((s, e))
49     return sorted(gts)
50
51
52     def make_predict_fn(trajecory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
53         """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
54         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
55
56         def predict_fn(cfg: Config) -> List[Interval]:
57             vt, mg, mwd = cfg
58             wins = TrackNetRunner.trajectory_to_action_windows(
59                 points, fps=fps, frame_width=frame_width,
60                 velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,

```

15. user (2026-06-08T10:23:16.783Z)

```

34: def load_golden_gts(csv_path: str) -> List[Interval]:
35-     """Rally [start,end] seconds from the golden CSV (start_time/end_time cols)."""
36-     gts: List[Interval] = []
37-     with open(csv_path, newline="", encoding="utf-8") as f:
38-         reader = csv.DictReader(f)
39-         reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or [])]
40-         for row in reader:
41-             row = {(k or "").strip().lower(): v for k, v in row.items()}
42-             try:
43-                 s = float((row.get("start_time") or "").strip())
44-                 e = float((row.get("end_time") or "").strip())
45-             except (TypeError, ValueError):
46-                 continue
47-             if e > s:
48-                 gts.append((s, e))
49-     return sorted(gts)
50-
51-
52- def make_predict_fn(trajecory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
53-     """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
54-     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
55-
56-     def predict_fn(cfg: Config) -> List[Interval]:
57-         vt, mg, mwd = cfg
58-         wins = TrackNetRunner.trajectory_to_action_windows(
59-             points, fps=fps, frame_width=frame_width,
60-             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
61-         )
62-         return [(w["start"], w["end"]) for w in wins]
63-
64-     return predict_fn
65-
66-
67- gts = load_golden_gts(labels_csv)
68- if not gts:
69-     raise SystemExit(f"no usable golden rallies in {labels_csv}")
70- predict_fn = make_predict_fn(trajecory_csv, fps, frame_width)
71- grid = default_grid()

```

```

81-
82-     base = evaluate(predict_fn(BASELINE), gts, iou)
83-     best_cfg, best_f1 = select_best(predict_fn, grid, gts, metric="f1", iou_threshold=iou)
84-     fit = improvement_report(predict_fn, BASELINE, best_cfg, gts, iou) # OPTIMISTIC (fit on
full GT)
85-     cv = cross_validate(predict_fn, grid, gts, k=5) # HONEST held-out
recall
86-
87-     print("=== WASB rally-segmentation calibration ===")
88-     print(f"golden rallies: {len(gts)} | IoU thresh: {iou}")
89-     print(f"baseline cfg {BASELINE}: precision={base.precision:.3f} recall={base.recall:.3f}
"
90-           f"f1={base.f1:.3f} meanIoU={base.mean_iou:.3f}")
91-     print(f"tuned cfg {best_cfg}: f1={best_f1:.3f}")
92-     print(f" precision {fit['baseline']['precision']:.3f} -> {fit['tuned']['precision']:.3f}
"
93-           f"({fit['precision_delta_pct']:+.1f}% , fit-on-full = optimistic)")
94-     print(f" f1 {fit['baseline']['f1']:.3f} -> {fit['tuned']['f1']:.3f} "
95-           f"({fit['f1_delta_pct']:+.1f}%)")
96-     print(f"HONEST (cross-val, {cv.get('folds','?')}) folds: held-out recall "
97-           f"{cv.get('mean_test',float('nan')):.3f} ± {cv.get('std_test',0):.3f}, "
98-           f"generalization_gap {cv.get('generalization_gap',float('nan')):+.3f}")
99-     print("(For precision/F1 generalization, add a 2nd labelled video ->
leave_one_video_out.)")
100-     return {"baseline": base.as_dict(), "best_cfg": best_cfg, "fit": fit, "cv": cv}
101-
102-
103-def main():
104-     ap = argparse.ArgumentParser()
105-     ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
106-     ap.add_argument("--labels", required=True, help="golden labels CSV (start_time/end_time
cols)")

```

16. assistant (2026-06-08T10:23:23.975Z)

`load\_golden\_gts` confirmed ? reads only `start\_time`/`end\_time`. Now let me verify the doc claims about `export\_eval\_set`/`export`, `CurationStatus` enum values, and where `import-local` sets the hardcoded `CURATED`. Let me also check `db.py::insert\_badminton\_rallies` for the DELETE-then-INSERT and PK behavior.

17. user (2026-06-08T10:23:25.955Z)

```

C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:117:
status=CurationStatus.CURATED, # Custom local imports bypass staging by default
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:359:           statuses =
[CurationStatus.CURATED, CurationStatus.STAGING]
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:450:           By default only
CURATED + commercially-safe videos are exported, mirroring
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:455:           statuses =
[CurationStatus.CURATED]
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:457:
statuses.append(CurationStatus.STAGING)
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:660:           staging_videos =
self.db.get_videos_by_status(CurationStatus.STAGING)
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:665:           print(f"\nFound
{len(staging_videos)} matches in STAGING state for curation:")
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:741:           print(" [a]
Approve (promote to CURATED)")
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:748:
self.db.update_status(video.video_id, CurationStatus.CURATED)
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:749:
print(f"Successfully promoted '{video.match_name}' to CURATED.\n")
C:\Users\avidu\Projects\sports-data-collector\src\reporting\adapter.py:42:
"act only if confirmed.")

```

```

C:\Users\avidu\Projects\sports-data-collector\src\parsers\badminton\coachai.py:145:
status=CurationStatus.STAGING,
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\base.py:24:class
CurationStatus(str, Enum):
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\base.py:25:     STAGING =
"staging"
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\base.py:26:     CURATED =
"curated"
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\base.py:27:     REJECTED =
"rejected"
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\base.py:37:     status:
CurationStatus = CurationStatus.STAGING

```

18. user (2026-06-08T10:23:27.839Z)

```

17:     SportType.BADMINTON: ("badminton_rallies", "rally_number"),
20:     SportType.PICKLEBALL: ("pickleball_rallies", "rally_number"),
21:     SportType.TABLE_TENNIS: ("tabletennis_rallies", "rally_number"),
45:     # ON DELETE CASCADE declarations on the child tables actually fire.
59:         video_id TEXT PRIMARY KEY,
77:         rally_number INTEGER NOT NULL,
85:         PRIMARY KEY (video_id, rally_number),
86:         FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
101:         PRIMARY KEY (video_id, point_number),
102:         FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
119:         PRIMARY KEY (video_id, over_num, ball_num),
120:         FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
128:         rally_number INTEGER NOT NULL,
136:         PRIMARY KEY (video_id, rally_number),
137:         FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
145:         rally_number INTEGER NOT NULL,
153:         PRIMARY KEY (video_id, rally_number),
154:         FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
171:     Uses an UPSERT rather than INSERT OR REPLACE: REPLACE would DELETE the
179:     INSERT INTO videos (
210: def insert_badminton_rallies(self, rallies: List[BadmintonRally]) -> int:
217:     cursor.execute("DELETE FROM badminton_rallies WHERE video_id = ?", (video_id,))
221:     INSERT INTO badminton_rallies (
222:         video_id, rally_number, start_time, end_time,
227:         rally.rally_number,
245:     cursor.execute("DELETE FROM tennis_points WHERE video_id = ?", (video_id,))
249:     INSERT INTO tennis_points (
272:     cursor.execute("DELETE FROM cricket_deliveries WHERE video_id = ?", (video_id,))
276:     INSERT INTO cricket_deliveries (
301:     cursor.execute("DELETE FROM pickleball_rallies WHERE video_id = ?", (video_id,))
305:     INSERT INTO pickleball_rallies (
306:         video_id, rally_number, start_time, end_time,
311:         rally.rally_number,
329:     cursor.execute("DELETE FROM tabletennis_rallies WHERE video_id = ?",
(video_id,))
333:     INSERT INTO tabletennis_rallies (
334:         video_id, rally_number, start_time, end_time,
339:         rally.rally_number,
419:     ORDER BY rally_number
424:         rally_number=row['rally_number'],
488:     ORDER BY rally_number
493:         rally_number=row['rally_number'],
511:     ORDER BY rally_number
516:         rally_number=row['rally_number'],
560:     cursor.execute("DELETE FROM badminton_rallies WHERE video_id = ?", (video_id,))
561:     cursor.execute("DELETE FROM tennis_points WHERE video_id = ?", (video_id,))
562:     cursor.execute("DELETE FROM cricket_deliveries WHERE video_id = ?", (video_id,))
563:     cursor.execute("DELETE FROM pickleball_rallies WHERE video_id = ?", (video_id,))
564:     cursor.execute("DELETE FROM tabletennis_rallies WHERE video_id = ?",
(video_id,))

```

```
565:         cursor.execute("DELETE FROM videos WHERE video_id = ?", (video_id,))
```

19. user (2026-06-08T10:23:29.194Z)

```
175:         rally_number=_safe_int(obj.get("rally_number"), idx + 1),
176:         start_time=_safe_float(obj.get("start_time")),
188:         start_time=_safe_float(obj.get("start_time")),
200:         start_time=_safe_float(obj.get("start_time")),
211:         rally_number=_safe_int(obj.get("rally_number"), idx + 1),
212:         start_time=_safe_float(obj.get("start_time")),
223:         rally_number=_safe_int(obj.get("rally_number"), idx + 1),
224:         start_time=_safe_float(obj.get("start_time")),
242:         rally_number=_safe_int(row.get("rally_number"), idx + 1),
243:         start_time=_safe_float(row.get("start_time")),
255:         start_time=_safe_float(row.get("start_time")),
267:         start_time=_safe_float(row.get("start_time")),
278:         rally_number=_safe_int(row.get("rally_number"), idx + 1),
279:         start_time=_safe_float(row.get("start_time")),
290:         rally_number=_safe_int(row.get("rally_number"), idx + 1),
291:         start_time=_safe_float(row.get("start_time")),
440:     def export_eval_set(self, args):
444:         indexer's `start,end` ground-truth format (decimal seconds), plus a single
492:         writer.writerow(["start", "end"])
494:         writer.writerow([start, end])
725:         sample_data = [[r.rally_number, f"{r.start_time}s - {r.end_time}s",
r.score_after, r.shots_count] for r in rallies[:5]]
731:         sample_data = [[p.point_number, f"{p.start_time}s - {p.end_time}s",
p.game_score] for p in pts[:5]]
737:         sample_data = [[f"{d.over}.{d.ball}", f"{d.start_time}s -
{d.end_time}s", d.runs, "Wicket" if d.wicket else ""] for d in delivs[:5]]
767:         start_time = None
778:         rally = next((r for r in rallies if r.rally_number == rally_num), None)
780:         start_time = rally.start_time
794:         start_time = pt.start_time
810:         start_time = d.start_time
816:         if start_time is None or end_time is None:
820:         print(f"Playing {label} (start: {start_time}s, end: {end_time}s)...")
826:         # Create file:// URL with media fragment (browsers support #t=start,end
fragment)
827:         play_url = f"file:///{{abs_path.replace(os.sep, '/')}}#t={{start_time}},{{end_time}}"
831:         t_sec = int(start_time)
```

20. assistant (2026-06-08T10:23:32.828Z)

Let me verify the remaining doc claims. First, the export: the doc says `export` produces `manifest.json + per-video start,end CSV`, references `curate.py::export\_eval\_set`, and claims "export orders by it [rally_number]". Let me read `export\_eval\_set` (440-500) and confirm whether the export query orders by rally_number. Also verify the `import-local` status setting around line 117 and how shots_count column passes through.

21. user (2026-06-08T10:23:34.334Z)

```
440     def export_eval_set(self, args):
441         """Export curated annotations as indexer-ready eval/training sets.
442
443         Emits, per qualifying video, a `/s<video_id>.csv` in the
444         indexer's `start,end` ground-truth format (decimal seconds), plus a single
445         `/manifest.json` that pairs each CSV with its video file. The
446         manifest is the contract the downstream eval/training driver reads: the
447         indexer keys videos by *file MD5*, not by our `video_id`, so it must run
448         against the exact `local_path`/`video_url` recorded here.
449
450         By default only CURATED + commercially-safe videos are exported, mirroring
451         the license gate already enforced elsewhere in the toolchain.
```

```

452         """
453         out_dir = args.out_dir
454         commercial_only = not args.include_noncommercial
455         statuses = [CurationStatus.CURATED]
456         if args.include_staging:
457             statuses.append(CurationStatus.STAGING)
458
459         supported = self.config.get("supported_sports", ["badminton"])
460         sport_strs = [args.sport] if args.sport else supported
461
462         manifest = []
463         skipped_noncomm = 0
464         skipped_empty = 0
465
466         for sport_str in sport_strs:
467             try:
468                 sport = SportType(sport_str.lower())
469             except ValueError:
470                 print(f"Warning: skipping unknown sport '{sport_str}'.")
471                 continue
472
473             videos = []
474             for status in statuses:
475                 videos.extend(self.db.get_videos_by_status(status, sport))
476
477             for video in videos:
478                 if commercial_only and not video.is_commercial:
479                     skipped_noncomm += 1
480                     continue
481
482                 intervals = self.db.get_segment_intervals(video.video_id, sport)
483                 if not intervals:
484                     skipped_empty += 1
485                     continue
486
487                 sport_dir = os.path.join(out_dir, sport.value)
488                 os.makedirs(sport_dir, exist_ok=True)
489                 csv_path = os.path.join(sport_dir, f"{video.video_id}.csv")
490                 with open(csv_path, "w", newline="", encoding="utf-8") as f:
491                     writer = csv.writer(f)
492                     writer.writerow(["start", "end"])
493                     for start, end in intervals:
494                         writer.writerow([start, end])
495
496                 manifest.append({
497                     "video_id": video.video_id,
498                     "sport": sport.value,
499                     "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),

```

22. user (2026-06-08T10:23:35.292Z)

```

95         elif sport_str == "cricket":
96             is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
97         elif sport_str == "pickleball":
98             is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
99         elif sport_str == "tabletennis":
100             is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
101
102         if not is_valid:
103             print(f"Error: Annotations failed validation!")
104             for err in errors:
105                 print(f"  - {err}")
106             sys.exit(1)
107
108         # 3. Create Video Metadata

```

```

109         video_meta = VideoMetadata(
110             video_id=video_id,
111             sport=SportType(sport_str),
112             video_url=None,
113             local_path=args.video_path,
114             license_type=license_type,
115             license_url=None,
116             is_commercial=is_commercial,
117             status=CurationStatus.CURATED, # Custom local imports bypass staging by
default
118             match_name=match_name,
119             fps=args.fps,
120             resolution=args.resolution
121         )
122
123         # 4. Insert into Database
124         self.db.insert_video(video_meta)
125
126         count = 0
127         if sport_str == "badminton":
128             count = self.db.insert_badminton_rallies(annotations)
129         elif sport_str == "tennis":

```

23. user (2026-06-08T10:23:35.862Z)

```

405         with self._get_connection() as conn:
406             cursor = conn.cursor()
407             cursor.execute(
408                 "SELECT sport, status, COUNT(*) as cnt FROM videos GROUP BY status,
sport"
409             )
410             return [(r["sport"], r["status"], r["cnt"]) for r in cursor.fetchall()]
411
412     def get_badminton_rallies(self, video_id: str) -> List[BadmintonRally]:
413         rallies = []
414         with self._get_connection() as conn:
415             cursor = conn.cursor()
416             cursor.execute("""
417                 SELECT * FROM badminton_rallies
418                 WHERE video_id = ?
419                 ORDER BY rally_number
420                 """, (video_id,))
421             for row in cursor.fetchall():
422                 rallies.append(BadmintonRally(
423                     video_id=row['video_id'],
424                     rally_number=row['rally_number'],
425                     start_time=row['start_time'],
426                     end_time=row['end_time'],
427                     score_before=row['score_before'],
428                     score_after=row['score_after'],
429                     shots_count=row['shots_count'],

```

24. assistant (2026-06-08T10:23:39.620Z)

Now let me check `get\_segment\_intervals` to confirm the export ordering claim (doc says "export orders by it"), and verify the validator behavior the doc claims (`validate\_badminton\_rallies` checks only positive duration + no overlap). Let me also check whether `BadmintonRally` validates `end > start` itself, since the test claims the blank-end row "would abort" ? important for confirming the integration claim.

25. user (2026-06-08T10:23:41.121Z)

```

527:     def get_segment_intervals(self, video_id: str, sport: SportType) -> List[tuple]:
528-         """Return chronologically-ordered (start_time, end_time) tuples for a video,

```

```

529-         regardless of sport. This is the sport-agnostic view used to export
530-         indexer-ready evaluation/training labels.
531-
532-         Sports that have no annotation table yet (e.g. tabletennis, pickleball)
533-         yield an empty list rather than raising.
534-         """
535-         mapping = SEGMENT_TABLES.get(sport)
536-         if mapping is None:
537-             return []
538-         table, order_by = mapping # both come from the trusted map above, never user input
539-         with self._get_connection() as conn:
540-             cursor = conn.cursor()
541-             cursor.execute(
542-                 f"SELECT start_time, end_time FROM {table} "
543-                 f"WHERE video_id = ? ORDER BY {order_by}",
544-                 (video_id,),
545-             )
546-             return [(row["start_time"], row["end_time"]) for row in cursor.fetchall()]
547-

```

26. user (2026-06-08T10:23:42.244Z)

```

29:     def validate_badminton_rallies(self, rallies: List[BadmintonRally]) -> Tuple[bool,
List[str]]:
30-         """
31-         Validates badminton rallies:
32-         - Sorted chronologically.
33-         - Start time < End time (handled by Pydantic, but double checked).
34-         - No temporal overlap between sequential rallies.
35-         """
36-         errors = []
37-         if not rallies:
38-             return True, errors
39-
40-         # Sort by rally number to check progression
41-         sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
42-
43-         for i in range(len(sorted_rallies)):
44-             r = sorted_rallies[i]
45-             if r.start_time >= r.end_time:
46-                 errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")
47-
48-             if i > 0:
49-                 prev = sorted_rallies[i - 1]
50-                 if r.start_time < prev.end_time:
51-                     errors.append(
52-                         f"Rally overlap detected between Rally {prev.rally_number} "
53-                         f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
54-                         f"(starts at {r.start_time}s)"
55-                     )
56-
57-         return len(errors) == 0, errors
58-
59:     def validate_tennis_points(self, points: List[TennisPoint]) -> Tuple[bool, List[str]]:
60-         errors = []
61-         if not points:
62-             return True, errors
63-
64-         sorted_pts = sorted(points, key=lambda p: p.point_number)
65-         for i in range(len(sorted_pts)):
66-             pt = sorted_pts[i]
67-             if pt.start_time >= pt.end_time:
68-                 errors.append(f"Point {pt.point_number}: start ({pt.start_time}) >= end
({pt.end_time})")

```


27. assistant (2026-06-08T10:23:48.682Z)

The export `ORDER BY rally\_number` confirms the doc's "export orders by it" claim. The validator confirms "positive duration + no overlap." Now one important detail: the doc and module docstring claim `safe\_float\_required` RAISES on blank/reversed end and aborts the import. But `end <= start` (reversed) does NOT make `safe\_float\_required` raise ? it only raises on blank/missing. A reversed end is caught later by the validator (`sys.exit(1)`). Let me verify there's no Pydantic validation on `BadmintonRally` that raises earlier, and re-read the exact wording of the claims to judge accuracy.

Let me check the `BadmintonRally` schema for a validator and confirm the negative `start\_time` edge case for the normalizer (does the normalizer allow negative start? The validator/schema may reject it).

28. user (2026-06-08T10:23:49.866Z)

```
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tennis.py:1:from pydantic import
BaseModel, Field, model_validator
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tennis.py:7:    start_time: float
= Field(..., description="Start of point play in seconds")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tennis.py:8:    end_time: float =
Field(..., description="End of point play in seconds")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tennis.py:14:
@model_validator(mode='after')
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tennis.py:16:        if
self.start_time < 0:
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tennis.py:17:            raise
ValueError("start_time must be non-negative")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tennis.py:18:        if
self.end_time <= self.start_time:
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tennis.py:19:            raise
ValueError("end_time must be strictly greater than start_time")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tabletennis.py:1:from pydantic
import BaseModel, Field, model_validator
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tabletennis.py:7:    start_time:
float = Field(..., description="Start of rally in seconds (serve racket-ball contact)")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tabletennis.py:8:    end_time:
float = Field(..., description="End of rally in seconds (ball dead)")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tabletennis.py:15:
@model_validator(mode='after')
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tabletennis.py:17:        if
self.start_time < 0:
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tabletennis.py:18:            raise ValueError("start_time must be non-negative")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tabletennis.py:19:        if
self.end_time <= self.start_time:
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tabletennis.py:20:            raise ValueError("end_time must be strictly greater than start_time")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\cricket.py:1:from pydantic import
BaseModel, Field, model_validator
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\cricket.py:8:    start_time:
float = Field(..., description="Start of delivery play in seconds (bowler run-up)")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\cricket.py:9:    end_time: float
= Field(..., description="End of delivery play in seconds")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\cricket.py:16:
@model_validator(mode='after')
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\cricket.py:18:        if
self.start_time < 0:
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\cricket.py:19:            raise
ValueError("start_time must be non-negative")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\cricket.py:20:        if
self.end_time <= self.start_time:
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\cricket.py:21:            raise
```

```

ValueError("end_time must be strictly greater than start_time")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\pickleball.py:1:from pydantic
import BaseModel, Field, model_validator
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\pickleball.py:7:    start_time:
float = Field(..., description="Start of rally in seconds (serve contact)")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\pickleball.py:8:    end_time:
float = Field(..., description="End of rally in seconds (ball dead)")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\pickleball.py:15:
@model_validator(mode='after')
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\pickleball.py:17:        if
self.start_time < 0:
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\pickleball.py:18:
raise ValueError("start_time must be non-negative")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\pickleball.py:19:        if
self.end_time <= self.start_time:
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\pickleball.py:20:
raise ValueError("end_time must be strictly greater than start_time")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\badminton.py:1:from pydantic
import BaseModel, Field, model_validator
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\badminton.py:4:class
BadmintonRally(BaseModel):
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\badminton.py:7:    start_time:
float = Field(..., description="Start of rally in seconds")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\badminton.py:8:    end_time:
float = Field(..., description="End of rally in seconds")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\badminton.py:15:
@model_validator(mode='after')
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\badminton.py:17:        if
self.start_time < 0:
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\badminton.py:18:            raise
ValueError("start_time must be non-negative")
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\badminton.py:19:        if
self.end_time <= self.start_time:
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\badminton.py:20:            raise
ValueError("end_time must be strictly greater than start_time")

```

29. assistant (2026-06-08T10:23:56.542Z)

This is a key finding. `BadmintonRally` has a Pydantic `model\_validator` that raises on:

- `start\_time < 0` (negative start)
- `end\_time <= start\_time`

This means the module docstring/doc claim about the abort mechanism is partly inaccurate, AND there's a real correctness gap: the normalizer allows **negative `start\_time`** through (it only checks `start is None`, `end is None`, `end <= start`), which would make `BadmintonRally` raise during `\_parse\_local\_annotations` and abort the import ? exactly the failure mode the normalizer claims to prevent.

Let me trace `\_parse\_local\_annotations` precisely: is `BadmintonRally(...)` constructed inside the parse loop (so a raise aborts), and does the negative-start row survive normalization? The normalizer's only start check is `start is None`. A row like `start\_time=-5, end\_time=10` passes (`end > start`), gets written, then `BadmintonRally` raises `start\_time must be non-negative`. Let me confirm the construction is inside the try-less parse path and where exceptions propagate.

30. user (2026-06-08T10:23:57.413Z)

```

150         fallback_dir=os.path.dirname(os.path.abspath(args.annotations_path)),
151     )
152     if report_paths:
153         print(f" - Observability report: {report_paths.get('technical_md')}")
154         print(f" - Customer summary      : {report_paths.get('customer_md')}\n")
155
156     def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
List[Any]:

```

```

157         """Parses simple JSON or CSV annotations files."""
158         parsed_items = []
159         ext = os.path.splitext(filepath)[1].lower()
160
161         if ext == '.json':
162             with open(filepath, 'r', encoding='utf-8') as f:
163                 data = json.load(f)
164
165                 if not isinstance(data, list):
166                     raise ValueError("JSON file must contain a list of objects.")
167
168                 # Uses the same safe casts as the CSV path: a missing/blank required
169                 # time raises a clear ValueError (caught by the caller) instead of a
170                 # bare KeyError, and optional ints degrade to sensible defaults.
171                 for idx, obj in enumerate(data):
172                     if sport == "badminton":
173                         parsed_items.append(BadmintonRally(
174                             video_id=video_id,
175                             rally_number=_safe_int(obj.get("rally_number"), idx + 1),
176                             start_time=_safe_float(obj.get("start_time")),
177                             end_time=_safe_float(obj.get("end_time")),
178                             score_before=obj.get("score_before"),
179                             score_after=obj.get("score_after"),

```

31. user (2026-06-08T10:23:58.099Z)

```

60         if not os.path.exists(args.video_path):
61             print(f"Warning: Local video path '{args.video_path}' does not exist on
disk, but continuing registration.")
62
63         # Validate annotation file
64         if not os.path.exists(args.annotations_path):
65             print(f"Error: Annotations file '{args.annotations_path}' not found.")
66             sys.exit(1)
67
68         match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
69         video_id = args.video_id or match_name.lower().replace(" ", "_")
70
71         # Parse license
72         try:
73             license_type = LicenseType(args.license)
74         except ValueError:
75             print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown
(non-commercial).")
76             license_type = LicenseType.UNKNOWN
77
78         is_commercial = self.validator.check_license_commercial(license_type)
79
80         # 1. Parse Annotations
81         annotations = []
82         try:
83             annotations = self._parse_local_annotations(sport_str, video_id,
args.annotations_path)
84         except Exception as e:
85             print(f"Error parsing annotations: {e}")
86             sys.exit(1)
87
88         # 2. Run Validation
89         is_valid = True
90         errors = []
91         if sport_str == "badminton":
92             is_valid, errors = self.validator.validate_badminton_rallies(annotations)
93         elif sport_str == "tennis":
94             is_valid, errors = self.validator.validate_tennis_points(annotations)

```

32. assistant (2026-06-08T10:24:09.453Z)

Confirmed. The flow is:

1. `\_parse\_local\_annotations` constructs `BadmintonRally(...)` per row inside the loop (line 240). A Pydantic `model\_validator` raises on `start\_time < 0` OR `end\_time <= start\_time`.
2. The caller wraps the entire parse in `try/except Exception` ? `sys.exit(1)` (lines 82-86) ? so one bad row aborts the whole import. This confirms the "one bad row kills the entire video" abort mechanism, but via the **Pydantic validator**, not `safe\_float\_required` for the reversed-end case.

So the normalizer's negative-start gap is a real bug: a `start\_time = -5` row survives normalization (only `start is None` / `end is None` / `end <= start` are checked) and then aborts the import via `start\_time must be non-negative`. Let me confirm there's no negative-start handling anywhere in the normalizer and double check the `\_to\_seconds`/sort path doesn't reject it.

The normalizer code (lines 122-153) only drops on `start is None`, `end is None`, `end <= start`. A negative start with valid end passes all three. Confirmed bug.

Now let me verify two more things: (1) the `shots\_count` column in the output ? does it pass through `\_safe\_int` safely (e.g. if it contains text)? and (2) the test's claim that blank-end "would abort" ? that's actually true via `safe\_float\_required` raising on blank. Let me also double-check the doc's "14-col" rich CSV claim and the `restrict\_to\_window` / `max(end\_time)` claims in the indexer, which I've now read.

Let me verify whether negative start_time is even reachable from the real annotator (rally-annotator monotonic), to calibrate severity, and check the test file doesn't cover negative start.

33. user (2026-06-08T10:24:11.313Z)

```
21:         {"rally_number": "2", "start_time": "20.0", "end_time": ""},          # blank end ->
drop
22:         {"rally_number": "3", "start_time": "30.0", "end_time": "28.0"},    # end<=start ->
drop
40:     # contiguous integer PKs in start order -> no (video_id, rally_number) collision
85:     with open(dirty, "w", newline="", encoding="utf-8") as f:
```

34. assistant (2026-06-08T10:24:16.912Z)

No negative-start coverage. The task explicitly listed "negative start_time" as an edge case to trace ? and it's a genuine hole. Let me confirm by quickly running the normalizer logic mentally is right, but better: let me actually execute the normalizer + parser against a negative-start row to prove the abort. Let me write a quick repro.

35. user (2026-06-08T10:24:22.354Z)

```
rows_out: 2 dropped: 0
out[0]: -5.000 10.000
PARSER RAISED -> ValidationError 1 validation error for BadmintonRally
  Value error, start_time must be non-negative [type=value_error, input_value={'vi
```

36. assistant (2026-06-08T10:24:28.086Z)

Confirmed empirically. The negative-start row passes normalization (`rows\_out: 2, dropped: 0`) and then the real `\_parse\_local\_annotations` raises a `ValidationError` ("start_time must be non-negative"), which the `import-local` caller turns into `sys.exit(1)` ? the exact whole-import abort the normalizer exists to prevent. This is a genuine landmine the normalizer fails to defend against.

Let me now verify the remaining doc claim I haven't directly confirmed: the "rich 14-col CSV" count and the `load\_canonical\_csv` reference. These are lower-priority but the task asks to spot-check every code reference. Let me check `load\_canonical\_csv` exists.

37. user (2026-06-08T10:24:29.390Z)

```
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:440:    def
export_eval_set(self, args):
C:\Users\avidu\Projects\sports-data-collector\src\parsers\cvat\rally_cvat.py:424:def
load_canonical_csv(csv_path: str) -> List[RallyRecord]:
```

38. assistant (2026-06-08T10:24:35.774Z)

Both `load_canonical_csv` and `export_eval_set` exist as referenced. The doc references `export_eval_set` as the export function and the `export` subcommand ? consistent.

Let me verify one subtle integration claim: the test calls `SportsDataCLI._parse_local_annotations(None, "badminton", ...)` with `self=None`. The real signature is `_parse_local_annotations(self, sport, video_id, filepath)`. Passing `None` as `self` works only if `self` is never used in the badminton CSV path. I read lines 156-302 ? `self` isn't referenced in the CSV badminton branch. That's fine. The test comment is accurate.

Now let me double-check the doc's claim in the field-mapping table: "`ending_reason` ... Vocab matches canonical enum". Let me check if `ending_reason` is actually an enum or a free string in the schema, since that's a factual claim.

39. user (2026-06-08T10:24:36.056Z)

```
1      from pydantic import BaseModel, Field, model_validator
2      from typing import Optional
3
4      class BadmintonRally(BaseModel):
5          video_id: str
6          rally_number: int = Field(..., description="Chronological rally index in the match")
7          start_time: float = Field(..., description="Start of rally in seconds")
8          end_time: float = Field(..., description="End of rally in seconds")
9          score_before: Optional[str] = Field(None, description="Score before the rally (e.g.
'0-0')")
10         score_after: Optional[str] = Field(None, description="Score after the rally (e.g.
'1-0')")
11         shots_count: Optional[int] = Field(None, description="Number of strokes in the
rally")
12         ending_reason: Optional[str] = Field(None, description="Why the rally ended:
'winner' | 'forced_error' | 'unforced_error' | 'service_fault' | 'let' | 'other'")
13         last_shot_outcome: Optional[str] = Field(None, description="Where the last shot
landed: 'in' | 'net' | 'out' | 'n_a' (n_a for service_fault/let)")
14
15         @model_validator(mode='after')
16         def validate_timestamps(self) -> 'BadmintonRally':
17             if self.start_time < 0:
18                 raise ValueError("start_time must be non-negative")
19             if self.end_time <= self.start_time:
20                 raise ValueError("end_time must be strictly greater than start_time")
21             return self
22
```

40. assistant (2026-06-08T10:25:11.425Z)

`ending_reason` is `Optional[str]`, not an enforced enum ? the doc's "Vocab matches canonical enum" is loosely worded but the description does list a canonical vocabulary, so it's defensible (a soft documentation claim, not a code-contract claim). Not a bug ? `ending_reason` is free-text, so the normalizer folding a pipe-delimited value to its primary token is safe regardless.

I now have everything I need. Let me confirm the `shots_count` pass-through one more time: the normalizer can emit non-integer `shots_count` (e.g. it copies `str(r["shots_count"]).strip()` verbatim). If the input had `shots_count="abc"`, `_safe_int(row.get("shots_count"), None)` returns `None` ? safe, no abort. Confirmed safe via parsing.py logic. Extra columns (`original_rally_number`, `notes`) ? `DictReader` / `row.get` ignores unknown keys entirely.

Confirmed no breakage.

Let me also confirm the empty-CSV / header-only path doesn't crash: ``normalize_rows([])`` ?
``rep.rows_in=0`, `kept=[]`, `orig_seq=[]`, `new_seq=[]`, `renumbered = ([] != []) = False`,
writes header only. And `normalize_file` on a file with no header: `reader.fieldnames` could be
`None` ? handled by `or []`. Safe. The "abc" time field ? `_to_seconds` catches `ValueError` ?
`None` ? row dropped, not crashed. All confirmed safe.`

I have completed the full trace. Here are my findings.

Adversarial review: ``normalize_annotations.py`` PR

Findings

****1. [major]** Negative ``start_time`` is NOT defended against ? still aborts the whole import**
- ****File:**** ``C:/Users/avidu/Projects/sports-data-collector/src/cli/normalize_annotations.py``
(drop logic, lines 129-140)
- ****What:**** ``normalize_rows`` only drops rows on ``start`` is ``None``, ``end`` is ``None``, or ``end`` <=
``start``. It never checks ``start`` < 0. A row like ``start_time=-5, end_time=10`` passes all three
guards and is written to the output CSV.
- ****Why it's wrong:**** ``BadmintonRally`` has a Pydantic ``model_validator``
(``src/core/schemas/badminton.py:17``) that raises ``"start_time must be non-negative"``. That
construction happens inside ``_parse_local_annotations`` (``curate.py:240``), and the ``import-local``
caller wraps the whole parse in ``try/except Exception`` ? ``sys.exit(1)`` (``curate.py:82-86``). So a
single negative-start row aborts the entire video import ? the exact landmine this normalizer
claims to eliminate. I reproduced it: the negative-start row yields ``rows_out: 2, dropped: 0``,
and the real parser then raises ``ValidationError: start_time must be non-negative``. The task
explicitly listed "negative start_time" as an edge case, and it is uncovered by both the code
and the tests.
- ****Fix:**** Add a drop branch ``if start < 0:`` (reason ``"negative start_time"``) alongside the
existing checks, and add a regression test. Optionally clamp/round instead of dropping if a tiny
negative is expected from float math, but dropping is consistent with the existing posture.

****2. [minor]** Docstring/doc misattribute the reversed-end abort to ``safe_float_required``**
- ****Files:**** ``normalize_annotations.py`` module docstring (lines 7-9);
``docs/MANUAL_INGESTION_POSITIONING.md`` (lines 59-60).
- ****What:**** Both say a "blank **or ``end_time`` <=``start_time`` ** row makes ``safe_float_required``
RAISE." That is only true for the ****blank/missing**** case. ``safe_float_required``
(``parsing.py:22-29``) parses any non-blank float line, including a reversed end ? it does not
raise on ``end`` <=``start``. The reversed-end row actually aborts via the ``BadmintonRally`` Pydantic
validator (``end_time`` must be strictly greater than ``start_time``), not ``safe_float_required``.
- ****Why it's wrong:**** The stated abort mechanism is inaccurate for half the cases it claims.
Both failure modes still abort the import (so the normalizer's **behavior** is right), but the
named cause is wrong ? and it masks the real validator, which is the same code path that also
enforces the (undefended) negative-start rule in finding #1.
- ****Fix:**** Reword to "a blank/missing time makes ``safe_float_required`` raise; a reversed (or
negative) time makes the ``BadmintonRally`` Pydantic validator raise ? either aborts the whole
import."

****3. [nit]** Field-mapping table overstates ``ending_reason`` as an enforced enum**
- ****File:**** ``docs/MANUAL_INGESTION_POSITIONING.md``, line 88 ("Vocab matches canonical enum").
- ****What:**** ``ending_reason`` is ``Optional[str]`` (``badminton.py:12``), free-text with a documented
vocabulary in its ``Field(description=...)``, not an enforced ``Enum``. No validation rejects an
off-vocab value.
- ****Why it's borderline:**** "canonical enum" implies enforcement that doesn't exist. Harmless to
the pipeline (the field is dropped before export), so nit-level.
- ****Fix:**** Say "vocab matches the documented canonical set (free-text, not enforced)."

Things I checked that are correct (not listed as issues)

- Fractional/duplicate ids never reach output ``rally_number``: output is always ``enumerate(..., start=1)`` integers (lines 169-171); ``original_rally_number`` carries the raw id. Verified the
duplicate-truncation case (``7.1`/`7.9``) rennumbers to ``1,2``, avoiding the ``(video_id,`

rally_number)` PK collision and the DELETE-then-INSERT data loss in `db.py:217-227`.

- Blank/reversed-end rows are dropped, not kept. Empty CSV, header-only, `"abc"` time field, out-of-order rows, duplicate `(start,end)` all handled without crashing (`\_to\_seconds` swallows `ValueError`; sort is stable; `renumbered` flag computed from `orig\_seq != new\_seq`).
- Integration: output columns feed `\_parse\_local\_annotations` safely. Extra columns (`original\_rally\_number`, `notes`, `shots\_count`) are ignored by `row.get(...)`; a non-numeric `shots\_count` degrades to `None` via `\_safe\_int`, no abort. Renumbered contiguous ints avoid the PK collision. The end-to-end test (`self=None`) is valid because `self` is unused in the badminton CSV branch.
- Doc code references verified: `load\_golden\_gts` reads only `start\_time`/`end\_time` (`calibrate\_wasb.py:34-49`); export is `start,end` intervals-only via `export\_eval\_set` writing `["start","end"]` (`curate.py:492`); `CurationStatus` is exactly `STAGING/CURATED/REJECTED` (`base.py:24-27`); `import-local` hardcodes `CurationStatus.CURATED` (`curate.py:117`); export `ORDER BY rally\_number` (`db.py:419`, `543`); `restrict\_to\_window` and the `window\_end = max(end\_time)` default (`eval\_partial\_labels.py:54, 75-76`) both match the doc's P1 claim, including the "wrong when tagged past last rally" caveat.

****Verdict: fix-first.**** One real correctness hole (negative `start\_time` still aborts the import ? finding #1, with a one-line fix + test) plus a doc/docstring inaccuracy about the abort mechanism (#2). Everything else in the PR is sound.